

UNIVERSIDAD DE LOS ANDES

FACULTAD DE INGENIERÍA

Ingeniería de Sistemas y Computación

Proyecto de Grado 2026-20

QuickR

Mateo López Cuartas

November 10, 2026

Bogotá D.C., Colombia

Contents

1 Introduction **2**
1.1 Context 2
1.2 Motivation 3

1 Introduction

1.1 Context

Ever since the turn of the millenium, the internet has had a significant impact on global communication technologies. By the year 2000, 51% of global telecommunications networks were transmitted via the internet, a figure that rose to 97% by 2007 (Hilbert & López, 2011). This reliance on the internet has been steadily increasing over time, culminating in 2020 when quarantine and lockdown measures due to the pandemic made the internet an essential part of daily life for most people worldwide. This drastic change in people's lives led to an all-time high demand for applications that connect different individuals through digital means, such as Meta's Metaverse, the increased demand for virtual meeting software (Zoom, Google Meet, Microsoft Teams), and mass communication services like Discord and Slack.

It is in this context that the need to adapt these daily activities to a new digital environment arises; in this case, broadcasting and sharing multimedia content via streaming from one device (host) for others to view on their devices in real time, regardless of their geographical location. Media streaming became popular through formal or academic means such as Zoom and Microsoft Teams, while the more casual media streaming needs were dominated by platforms like Twitch and Discord (the latter, although primarily used as a social messaging platform, also offers VoIP and video streaming services, which is the relevant service for this paper). The rapid expansion of video traffic on the internet, coupled with the staggering amount of users of these services, has created a strong need to develop robust protocols that work quickly and efficiently under any type of connectivity conditions, as well as infrastructures that are sufficiently robust and scalable to meet the demand for these transmissions.

This has led to an interest by developers to restrict in certain ways some functionalities and features offered by these streaming services in a way that is actually profitable. Paywalls and subscription models are some of the most common ways companies and services implement a restriction that solves multiple problems at the same time: first of all, it limits the number of users that can use the service at any given moment, which reduces maintenance costs and bandwidth usage; second, it produces a source of income for the service provider, which can be used to improve the service and infrastructure; and third, it helps create a sense of exclusivity to the paying user. Some

services that include these kinds of subscriptions are Discord, Google Meet, and Zoom. In the case of Discord, the service offers a subscription called Nitro that, besides other benefits, allows the user to upgrade the stream quality from the free tier cap of 720p x 30fps to a maximum of 4k x 60fps, with a monthly cost of $9.99 USD (as of November 2026). With Google Meet, the free tier besides allowing a call for only up to 60 minutes, hard limits the stream quality to 720p x 30fps, while any of Google's subscriptions allows for a maximum of 1080p x 30fps. Zoom, on the other hand, does not really restrict the stream quality, allowing content to be shared in the native resolution at 30fps for no extra cost, but it does recommend downscaling to 1080p to save on bandwith. It also limits the free tier to 40 minutes per call, restricting the user not by the quality of the stream, but by the time they can use it.

Given the context of these service models with their constraints, it becomes apparent that users, even though there are multiple options for meeting their streaming needs, are still limited by the restrictions imposed by these companies. Open source alternatives to these services, although existing, are not as widespread and used as the commercial ones, which reduces the appeal to the average user. This situation gives developers both an intrinsic and extrinsic motivation to create a new service that improves over existing alternatives while also being open source and free of use.

1.2 Motivation

The protocol is inspired by RTP [1]. as

References

[1] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson, "RTP: A transport protocol for real-time applications," RFC 3550, 2003. [Online]. Available: `https://www.rfc-editor.org/rfc/rfc3550`